PaperDraw: Intelligent Generation System for Scientific Publications Pipeline Diagrams

Xin Li*, Jixiu Liao, Chenglin Xu and Xianzheng Pang

College of Computer Science and Technology, Qingdao Institute of Software, China University of Petroleum (East China), Qingdao, 266580, Shandong, China

ARTICLE INFO

Keywords:
Visualization Generations
Layout Optimization
Visual Style Recommendations
Authoring tool
Natural Language Processing

ABSTRACT

Pipeline diagrams in scientific publications effectively reduce readers' cognitive load and enhance article readability. However, manually constructing these diagrams is complex and time-consuming, highlighting the need for efficient automated generation methods. While existing automation methods can encode logical relationships, their rigid layouts and limited visual styles lack the flexibility and aesthetic polish required for publication. In this work, we present PaperDraw, a system that generates publication-quality pipeline diagrams from natural language descriptions. To this end, we introduce a controllable layout optimization algorithm, which translates insights from design space analysis into actionable rules to enforce academic visual standards. Furthermore, we construct a style vector library from the dataset to recommend a diverse set of visual style candidates based on structural similarity. Flexible user adjustments and customization effectively eliminate the need for tedious manual post-processing, allowing researchers to rapidly iterate and finalize a professionally styled diagram directly from their logical structure. We demonstrate PaperDraw's efficiency and effectiveness through case studies, user evaluations, and both qualitative and quantitative comparisons with existing methods. The results indicate that PaperDraw enables the efficient generation of pipeline diagrams that are both precise in semantics and compelling in visual design.

1. Introduction

As the dominant medium of academic communication, scientific publications are inherently complex and specialized, presenting a significant cognitive barrier to readers (Head et al., 2021). Tenopir et al. (2009) indicate that scholars spend hundreds of hours each year reading scientific publications. Dual-coding theory (Clark and Paivio, 1991) in cognitive psychology posits that visual information facilitates cognitive processing, reduces cognitive load, and improves comprehension and memory. Consequently, pipeline diagrams in scientific publications serve as an efficient textual visualization that significantly enhances readability. By integrating text and images, these pipeline diagrams allow readers to quickly and accurately grasp the logical connections of the research methodologies, bypassing the need for close reading (Alrwele, 2017; Gastel and Day, 2022).

However, creating a scientific publications pipeline diagram that is both semantically accurate and visually appealing remains a complex and time-consuming challenge for researchers (Horita et al., 2024). This challenge arises from two primary factors. First, these pipeline diagrams integrate diverse visual elements—icons, imagery, geometric shapes, and text—all of which must be precisely aligned with the underlying information. This demands considerable proficiency in visual design and information visualization techniques from the designer. Second, the spatial arrangement of these elements is not merely a structural concern but also critical for managing cognitive load and ensuring perceptual fluency. A practical layout enhances hierarchical and logical clarity, conveys implicit information, and ultimately improves the reader's perceptual efficiency (Lu et al., 2020).

Furthermore, researchers often possess merely a preliminary concept when drafting pipeline diagrams, posing a significant barrier to the effective use of granular authoring tools. Natural language, being the most intuitive mode of human interaction, is particularly well-suited to express vague intentions. Therefore, a method that derives logic information and interprets desired artistic effects from user-provided text to generate pipeline diagrams naturally aligns with innate human interaction instincts (Wang et al., 2023). There are already many studies on text-based visualization generation. For example, mainstream data-driven methods can generate statistical charts, infographics, and data stories. However, these methods often center around data charts or focus on narrative storytelling, making it challenging to express non-data concepts and complex logical processes in scientific publications. While automated flowchart generation techniques excel at conveying rigorous logical structures, their layouts tend to focus on basic topological rendering, resulting in a rigid form that fails to accommodate personalized design requirements. Additionally, these techniques lack visual emphasis on key semantic elements and stylized design, making it challenging to meet the standardized requirements of academic papers.

To address these challenges, we present PaperDraw, a system that leverages text and Q&A inputs to capture user intent and incorporates interactive mechanisms to efficiently generate semantically accurate, standardized pipeline diagrams for scientific publications. Specifically, we first analyze and explore the design space of 300 pipeline diagram

*Corresponding author
lix@upc.edu.cn (X. Li); liaojxiu@163.com (J. Liao); chenglinxu@s.upc.edu.cn (C. Xu); 2307030124@s.upc.edu.cn (X. Pang)
ORCID(s): 0009-0003-9908-8910 (X. Li); 0009-0006-3408-0290 (J. Liao); 0009-0007-4012-2298 (C. Xu); 0009-0004-0894-2921 (X. Pang)

examples collected from scientific publications, summarizing a set of design principles and academic standards. Based on these principles, we develop a controllable layout optimization algorithm that translates insights from our design space analysis into actionable rules for enforcing academic visual standards. When the system captures the user's design intent using large language model (LLM) prompt engineering, it automatically generates the corresponding pipeline layout based on the algorithm. Furthermore, we construct a style vector library from a curated dataset of academic diagrams, which enables the system to automatically recommend a diverse set of visual style candidates with high structural similarity. Users can then make a selection, and the system adaptively applies the chosen style through prompt engineering. PaperDraw also integrates high-level user interaction directly into the optimization loop, allowing flexible adjustments and customization, thus eliminating the need for tedious manual post-processing and enabling researchers to quickly iterate and finalize a professionally styled diagram directly from their logical structure.

We evaluate PaperDraw's effectiveness through a comparative study on a dataset of collected pipeline diagrams from scientific publications, employing both quantitative metrics and qualitative analysis. The evaluation focuses on two dimensions: information content and visual perception. In summary, the significant contributions of this paper are as follows:

- A novel framework for automatically generating publication-quality pipeline diagrams that combine semantic accuracy with visual appeal from natural language descriptions, grounded in a design space analysis and the resulting design principles.
- A controllable layout optimization algorithm that combines design principles to produce pipeline diagram layouts. These layouts accurately reflect logical flows and adhere to academic standards in both entity arrangement and conduit routing.
- An efficient connection routing algorithm that accelerates layout computation to facilitate an iterative design process with real-time interaction and immediate feedback.
- A data-driven style vector library that recommends diverse visual style candidates for pipeline diagrams based on structural similarity measurements and applies them adaptively through prompt engineering.

2. Related Work

2.1. Auto-Generated Visualizations

Automated visualization generation is a significant field at the intersection of information visualization and intelligent design, aiming to convert data or semantic descriptions into visual representations. Although generation targets vary, existing research primarily revolves around key dimensions such as data mapping, logical presentation, and visual expression, spanning the following aspects:

In data-driven visualization generation, research is predominantly concerned with producing standard statistical charts from text or data. For instance, both end-to-end neural networks (Dibia and Demiralp, 2019) and methods leveraging LLMs (Dibia, 2023) can efficiently translate data summaries or textual commands into chart specifications, thereby facilitating an accurate representation of data relationships. However, a key limitation of these approaches lies in their restricted visual vocabulary, wherein the standard chart serves as the fundamental 'atomic unit'. Consequently, the design space is confined by data-encoding conventions, which hampers their ability to convey complex logical flows, qualitative conceptual relationships, or methodological innovations prevalent in academic discourse.

Infographics (Zhu et al., 2020) and data stories embody more complex narrative structures. Such work seeks to organically integrate multiple visual elements with textual annotations, forming a coherent visual narrative. For example, systems like PlotThread (Tang et al., 2021) can automatically organize event-based text into a sequential infographic to enhance storytelling. These studies provide valuable insights for automated layout and the composition of multiple elements. Nevertheless, these methods typically depend on well-structured inputs, and their visual language and narrative logic are primarily tailored for communication to a broad audience. As a result, they often fall short in capturing domain-specific conventions and representing nuanced logical hierarchies.

From the perspective of underlying task similarity, the automated generation of flowcharts shares the closest affinity with our work, as both focus on representing logical relationships among entities. Prior research has demonstrated the generation of business process diagrams, UML diagrams (Kulkarni et al., 2023), and even industrial PID diagrams (Gowaikar et al., 2024; Merkelbach et al., 2024) While these techniques successfully convert text into basic topological structures, the resulting diagrams typically comprise only primitive geometric shapes and connectors, relying on rigid, rule-based layouts. In essence, they constitute simple renderings of topology, wherein layout algorithms are primarily designed to minimize connection crossings rather than to optimize visual hierarchy. Crucially, existing techniques cannot evaluate the semantic importance of key nodes and consequently lack the mechanisms for visual emphasis and stylized design expressiveness. However, these capabilities are essential for scientific publications pipeline diagrams, which highlight technical contributions and articulate innovation pathways. In summary, existing methods are limited by shortcomings in formal style, logical rigor, and narrative expressiveness. This study, therefore, aims to transcend these limitations by proposing a novel paradigm that simultaneously addresses deep logical comprehension, the identification of salient semantics, and advanced visual design. Our ultimate objective is to intelligently generate

high-quality pipeline diagrams that are both rigorous and expressive for scientific publications.

2.2. Natural Language Interaction

In recent years, natural language has emerged as a promising interface for visual data analysis, enhancing the usability of visualization systems. Commercial tools like IBM Watson Analytics, Microsoft Power BI, and Tableau automatically translate natural language questions into data queries and visually present results. Systems such as Eviza (Setlur et al., 2016) employ various methods to transform natural language queries into formal database queries (e.g., SQL) to extract analytical results. However, these systems confine natural language interaction to data queries and standard chart generation. Analysis-oriented natural language interfaces (NLIs) generate one-off visualizations to address user analytical tasks, encompassing only a subset of the editing operations required for full-fledged visualization authoring. In creation-oriented NLI pipelines, by contrast, user editing intents are modeled as editing actions; these actions serve as an intermediate layer that decouples the natural language interpreter from the visualization application (Wang et al., 2023).

The advent of LLMs, renowned for their superior natural language processing and visual communication capabilities, has spurred numerous studies on LLM-powered visualization generation (Li et al., 2025; Cheng et al., 2025). For instance, the LIDA system by Dibia et al. (Dibia, 2023) leverages LLMs to enable grammar-agnostic generation of visualizations and infographics. He et al. (He et al., 2025) summarized specific tasks in the visualization generation pipeline where LLMs can be effectively applied and proposed a framework for leveraging LLMs throughout the narrative visualization process. Systems such as Socrates (Wu et al., 2024) and NL2Color (Shi et al., 2024) leverage large language models to automatically generate story flow diagrams and chart color schemes from natural language inputs. However, even with the capabilities of these advanced models, it remains challenging to produce satisfactory pipeline diagrams for scientific publications directly from natural language.

2.3. Automatic Layout Optimization

Layout tasks are ubiquitous in design applications, spanning magazine covers, posters, presentation slides, and application user interfaces. Existing automatic layout methods can be broadly categorized into template-based, optimization-based, and generative model-based approaches.

Template-based layout generation methods (Cui et al., 2020) rely on a predefined and limited library of templates. This approach can provide rapid solutions for scientific publications pipeline diagrams by mapping elements to fixed positions, achieving a basic layout. However, this approach suffers from limited flexibility and is inadequate for diagrams with numerous visual elements or complex structures. Moreover, they offer limited interactivity and fail to effectively capture or convey nuanced user design intent, which is crucial for highlighting specific research logic and key contributions. To overcome the rigidity of template-based methods, optimization-based approaches (Moritz et al., 2019) formulate layout as a constrained numerical optimization problem. These methods typically require the manual specification of complex constraints and a cost function that quantifies violations of aesthetic criteria. An optimal layout is then identified by algorithmically searching for the configuration that minimizes this cost. For example, Liu et al. (Shixia Liu et al., 2013) framed storyline layout as an optimization problem based on narrative metrics. However, their algorithm is primarily designed for arranging lines and ignores other visual elements.

In recent years, generative models, including Generative Adversarial Networks (GANs) (Inoue et al., 2023a), Transformers, and Diffusion Models (Inoue et al., 2023b), have introduced data-driven approaches to layout generation. For instance, LayoutGAN (Li et al., 2021) transforms the task into an image generation problem via a differentiable wireframe rendering layer. At the same time, BLT (Kong et al., 2022) employs a bidirectional Transformer architecture to support flexible conditional generation through a "fill-in-the-blank" task. Although these models demonstrate strong generative capabilities in general design tasks, their process often prioritizes statistical co-occurrence patterns of visual elements over the strict sequential logic and hierarchical relationships critical in academic diagrams. Furthermore, as these models are typically trained on generic design datasets, the resulting visual styles often fail to satisfy the professionalism, conciseness, and standardization mandated in academic publishing. Therefore, developing layout algorithms that seamlessly integrate semantic logic while adhering to a rigorous logical structure remains an open and significant research challenge.

3. Design of the PaperDraw System

3.1. Preliminary Survey

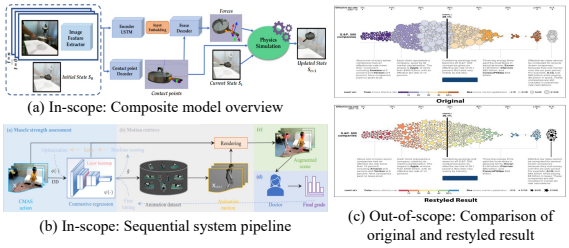
3.1.1. Dataset

This survey aims to characterize the application patterns of pipelines in scientific publications. Based on these patterns, we define a design space for pipeline construction. Through a comprehensive review of the literature and the inherent characteristics of pipelines, we establish the following inclusion criteria:

- It outlines the paper's primary methodology or process.
- It comprises multiple distinct visual elements.
- It adopts a flowchart-like structure that expresses logical relationships between the elements.

These criteria were applied to systematically identify and collect pipelines that align with our methodological definition. For instance, Fig. 1(a)(Ehsani et al., 2020) and Fig. 1(b)(Zhou et al., 2023) exemplify pipelines that conform to our definition, incorporating both intent and procedural

details. In contrast, Fig. 1(c)(Harper and Agrawala, 2014), although reflective of the study's objective, was excluded due to the absence of explicit procedural information. Our dataset was compiled from a corpus of 300 papers published between 2020 and 2024 in top-tier journals, including CVPR and TVCG. We identified 98 distinct pipelines from this corpus that satisfied all three inclusion criteria.



(a) In-scope: Composite model overview

(b) In-scope: Sequential system pipeline

(c) Out-of-scope: Comparison of original and restyled result

Fig. 1: Illustration of pipeline selection based on definitional criteria: (a, b) Examples of pipelines that include both composition and workflow; (c) A pipeline excluded due to lack of procedural information.

3.1.2. Statistics

We statistically analyzed the collected pipeline dataset, characterizing the distributions of module counts, visual element types, and layout forms. Fig. 2 presents a structured overview of these distributions through a grid of plots. The off-diagonal cells show co-occurrence counts between pairs of attributes (color-coded), while the diagonal cells display histograms for each attribute's value distribution.

For "module count", diagrams with 1, 2, or 4 modules are most common. These primarily adopt vertical or horizontal layouts. In contrast, alternative layouts (e.g., triangular, circular) are associated with a higher module count, although pipelines with more than five modules constitute a minority of the dataset.

Analysis of the "element type" distribution reveals that rectangular elements are predominant, with cubes being the second most common. Furthermore, the prevalence of rectangular and cubic elements increases with the total number of modules, a trend not observed for other element types. This suggests that other element types, when present, typically appear in small numbers (often just one or two per diagram), likely representing specialized entities.

Finally, the "layout" analysis reveals that the horizontal layout is the most frequent, followed by the vertical layout; all other layout types are comparatively rare. We posit that this hierarchy is primarily determined by space efficiency. A horizontal layout typically minimizes redundant space on a page, conforming to the rectangular geometry of a manuscript. The secondary prevalence of vertical layouts can often be explained by their fit within one column of a two-column template. A further practical consideration is that page limit constraints and publishing costs disincentivize the use of inefficient, irregular layouts that consume more space.

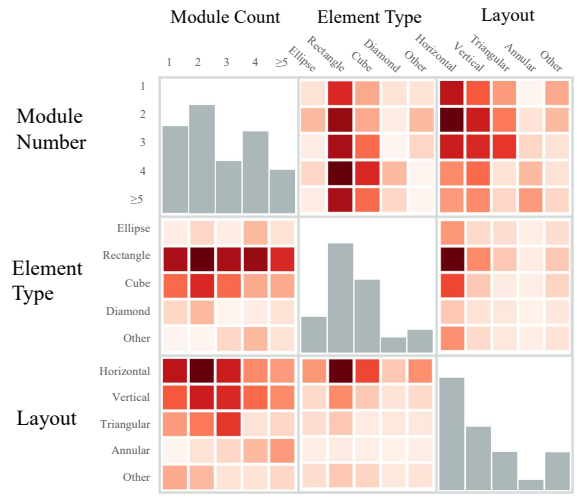


Fig. 2: Statistical findings from the dataset.

Beyond the dataset analysis, we synthesize design guidelines from leading scientific journals (e.g., Nature) and established visualization literature (Tang et al., 2021; Cui et al., 2020; Moritz et al., 2019)to derive common design principles for pipeline optimization. These principles are outlined as follows:

P1: Maintain Element Alignment. Visual elements should be systematically aligned to create layout coherence. For instance, elements within the same row should share a common baseline or be center-aligned.

P2: Maximize Space Efficiency. Layouts should utilize the available space effectively, minimizing excessive white space through compact and logical arrangements to achieve high information density.

P3: Adhere to Standardized Dimensions and Scaling. The overall pipeline should fit a rectangular bounding box that is compliant with single or double-column formats. Single-column figures should not exceed 240 pt in width, while double-column figures must range between 300 and 504 pt. The maximum height for any figure, including its caption, is 660 pt. At these dimensions, all graphical objects must scale to between 6 pt and 12 pt, with uniform sizing for all labels, letters, and symbols within a figure.

P4: Optimize Connection Routing. Connection crossings should be minimized. Arrows should primarily follow a left-to-right and top-to-bottom flow to align with natural reading patterns and facilitate intuitive visual tracking.

P5: Ensure Consistent Color Encoding. A harmonious color scheme should be applied consistently, using identical colors for elements that share the same hierarchical level or semantic meaning to reinforce the information structure and aid interpretation.

3.2. Design Space

Leveraging findings from an initial survey, we further analyze the design space of pipeline diagrams and formalize a taxonomy comprising four key aspects: layout, visual elements, relationships and color. Furthermore, based on

statistical data analysis, we develop a structured composition model for scientific publications pipeline diagrams.

3.2.1. Layout

Our analysis first identifies three fundamental spatial layout types as the foundational building blocks: horizontal, vertical, and diagonal. These base patterns are systematically organized into the standardized taxonomy presented in Fig. 3. To construct more complex layouts comprising numerous elements, we employ a recursive composition strategy that iteratively combines these structural units. Beyond the arrangement of functional elements, the routing and organization of connecting lines constitute another critical dimension in pipeline construction.

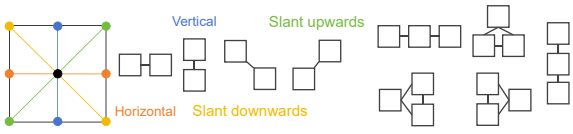


Fig. 3: A taxonomy of base patterns and derived paradigms.

3.2.2. Visual Elements

Shape representation in pipelines addresses two primary considerations. First, graphical elements must maintain strong semantic alignment with their corresponding textual descriptions. Second, different shapes serve distinct communicative functions. Our analysis of the pipeline dataset yields a taxonomy of common graphical elements—ellipses, rectangles, three-dimensional shapes, diamonds, and other specialized forms—with rectangles emerging as the most universally employed shape type.

Description serves as another core element, with the original narrative text providing the foundational material. Designers typically extract key information and arrange it into text blocks, which collectively convey the complete information when combined within a pipeline. Textual descriptions are applied in three principal ways: as entity labels placed on or near shapes to identify an element or its state; as action descriptors positioned along connecting lines to specify an operation or transition; and as explanatory notes, often in the form of concise phrases, that provide additional context or clarification.

3.2.3. Relationship

Element relationships in pipeline diagrams are classified into three primary categories: sequential, modular, and annotative(Fig. 4). Sequential relationships are conventionally represented by directed arrows, clearly indicating the order or dependency between elements. Modular relationships are commonly expressed using bounding boxes to encapsulate functional units or procedural steps, aiding in the comprehension of complex structures. Annotative relationships are typically rendered with dashed or branching lines, effectively conveying hierarchical or affiliative connections between components.

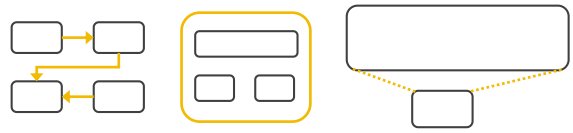


Fig. 4: Sequential, Modular, and Annotative Relationships.

3.2.4. Color

Each pipeline adheres to a coordinated color scheme, with deliberate color assignment across its components. Through stylistic feature extraction and semantic analysis of the dataset, we identify the following design principles:

- **Law of Similarity:** Elements within the same module share a color family, while those at an equivalent hierarchical level or with identical semantics are often assigned the same color.
- **Background and Foreground:** The background color, occupying a substantial area, should contrast distinctly with foreground elements. This is achieved through complementary colors or varying brightness levels within the same hue. Backgrounds are generally light-filled or outline-only, ensuring overall color balance and readability. Variations in background color also help visually separate different modules.
- **Highlight:** Color can serve as an emphasis mechanism, drawing attention to key elements or modules through relatively brighter, darker, or more saturated tones. In pipeline diagrams, the highlight is strategically applied to specific modules, directing the reader's focus to emphasize the paper's novel contributions and key findings, thereby enhancing the diagrams' explanatory power.

3.3. Design Considerations

The principal objective of the proposed system is to leverage an LLM coupled with a conversational agent (CA) for semantic extraction and user intent capture. This approach establishes logical constraints for pipeline design and enables iterative optimization under defined design principles. Building upon the prior dataset analysis and design space exploration, we distill four core design considerations for an intelligent generation system of scientific publications pipeline diagrams:

D1: Data Processing Layer for Entity Recognition and Relationship Extraction. This layer is designed to perform named entity recognition and extract in-depth relational structures. It must accurately interpret relationships among entity elements, with particular attention to text-image alignment. For user-provided descriptions of the paper framework, the system extracts predefined entity types—including core elements and procedural relationships—and further infers implicit information such as element importance weights and modular organization. The extraction process necessitates a rigorous assessment of logical consistency and semantic coherence in the text,

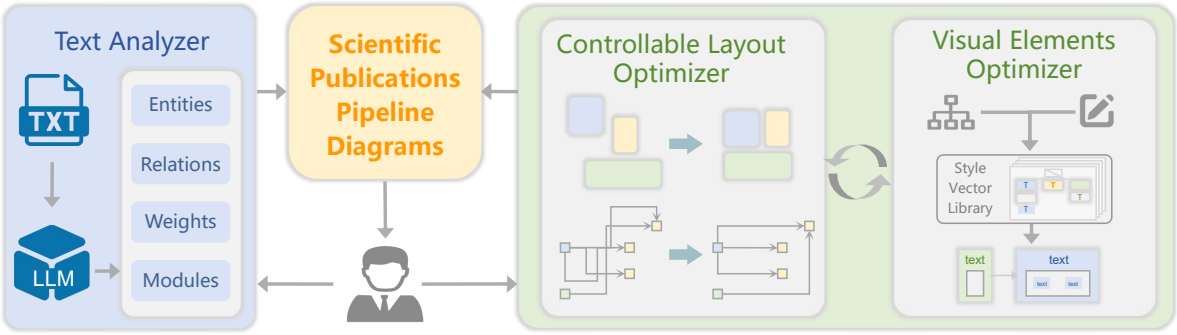


Fig. 5: Our framework comprises three key components: (1) a Text Analyzer that parses user input to derive structural information; (2) a Controllable Layout Optimizer that employs multi-objective optimization to arrange elements and routes connections; and (3) a Visual Element Optimizer that retrieves the best-matching style from a library and performs self-adaptive adjustments.

ensuring that the resulting diagram exhibits high semantic completeness and accuracy.

D2: Logic Layer for Semantic-Aware Layout and Visual Element Design. This layer constructs layout and visual element configurations that adhere to semantic constraints. In addition to spatial arrangement and perceptual effectiveness, it integrates Gestalt principles (Desolneux et al., 2004) with semantic information to derive visual constraints for shapes, text, icons, and spatial layout. For example, based on the law of similarity, color constraints are formulated by combining color semantics and the structural relationships among elements. Using the principle of proximity along with element hierarchies, the system reinforces hierarchical and logical expression in the layout. The outcome is a spatial constraint strategy that achieves high semantic consistency and perceptual effectiveness, ensuring close alignment between visual structure and semantic meaning.

D3: Visual Layer for Interactive Optimization of Layout and Visual Elements. This layer frames layout optimization as a multi-objective problem aimed at minimizing four key metrics: connection crossings, deviation from an ideal bounding-box proportion, blank area, and visual incoherence. By integrating semantic constraints associated with visual elements, it systematically accounts for the influence of spatial arrangement on overall visual effect. The outcome is a set of visual elements that achieve both semantic alignment and perceptual harmony.

D4: Interaction and Evaluation Layer for Iterative User-Driven Refinement. Existing tools for visual data storytelling struggle to balance efficiency and customizability. While highly customizable authoring tools (Lee et al., 2015) often demand substantial manual effort during insight discovery and narrative construction, fully automated pipelines (Shi et al., 2021b,a) offer limited support for integrating user intent into generated stories (Wu et al., 2024). To address this gap, we introduce an interaction and evaluation layer into the automated generation workflow. This allows users to supervise and guide each step of the pipeline generation, ensuring their design intent is effectively incorporated throughout the iterative refinement process.

4. Approach

4.1. Overview

As depicted in Fig. 5, the system begins by processing user-provided textual input through an LLM in conjunction with the CA to extract multi-dimensional information (e.g., entity nouns, procedural relationships, information importance weights, and module structures). This information is visualized using a predefined code framework, allowing users to semantically evaluate and interactively adjust the results, thereby ensuring the accuracy and completeness of the LLM-extracted information.

The extracted information is then fed into a controllable layout optimizer. This module employs a multi-objective optimization algorithm to iteratively refine the arrangement of elements, alongside a connection routing algorithm to optimize connection paths. The optimized layout is subsequently passed to a visual elements optimizer, where graphical components are adapted based on their semantics, with style recommendations retrieved from a style vector library.

The generated pipeline diagram is finally delivered to the user for evaluation. Users may further personalize the layout and visual style through interactive adjustments, which are incorporated as hard constraints in subsequent optimization runs. This process iterates based on continuous user feedback until the output aligns with the user's expectations.

4.2. Text Analyzer

This analyzer is designed to extract entity keywords and their in-depth associative relationships from a textual description of a scientific methodology. Furthermore, to ensure the resulting pipeline diagram accurately aligns with user intent, the CA is incorporated to capture user intent, which is primarily manifested as the modular relationships and importance weights among entities (D1). Finally, leveraging a predefined code framework, the extracted information is used to generate a textual visualization. Users can then perform a semantic evaluation and interactively modify the information through this output.

4.2.1. Textual Information Extraction

The extraction process begins with user-provided text. Using few-shot prompting, an LLM is guided to extract structured information, which is formatted for subsequent flowchart generation. The detailed procedure is as follows:

Given an input text sequence $T = \{t_1, t_2, ..., t_n\}$, the LLM encodes the text to generate contextualized semantic representations $H = \{h_1, h_2, ..., h_n\}$. A named entity recognition module then identifies a set of key entities $E = \{e_1, e_2, ..., e_m\}$ from the text based on these contextualized semantic representations H, where e_i denotes the i named entity. For each identified entity pair (e_i, e_j), semantic analysis is performed to derive a corresponding relationship vector, resulting in a set of deep semantic relationships $R = \{r_{ij} | e_i, e_j \in E\}$, where r_{ij} denotes the relationship between entities e_i and e_j.

4.2.2. Conversational Agent for Intent Refinement

Automated extraction by the LLM often falls short in generating nuanced scientific publications pipeline diagrams, as textual descriptions typically lack explicit modular structures and relative importance among entities—both critical for coherent visualization. Moreover, purely text-based input may inadequately convey user intent, frequently resulting in generic outputs. To address these limitations, this work introduces the CA for iterative intent refinement, drawing on methodologies from Conversational Recommendation Systems (CRSs), which have demonstrated efficacy in dynamically capturing user preferences (Wu et al., 2024).

Integrated within the text analyzer, the proposed CA engages the user in a structured, multi-turn dialogue. It operates through a two-step iterative cycle: First, it adaptively generates contextual questions based on the initial text analysis to elicit user feedback on modular groupings and entity weighting. Second, it processes this feedback to refine and disambiguate the extracted information. This process is guided by a constrained interaction protocol that defines both the agent's question types (e.g., "Is 'information generation' a core component of this paragraph?" or "Which entities belong to the same module?") and the user's available response options.

By incorporating explicit user choices as supervisory signals, the CA significantly enhances the extraction of modular relationships and importance weights. This interactive refinement ensures that the final output—comprising a well-defined module structure and entity-weighted graph—accurately reflects the user's conceptual framework and design intent.

4.3. Controllable Layout Optimizer

The layout comprises two core components: element arrangement and connection routing, each optimized by a dedicated algorithm. To effectively integrate user intent into the optimization process, we introduce three interaction modes that allow user preferences and requirements to be explicitly incorporated into the final layout.

4.3.1. Element arrangement optimization

First, logical constraints are constructed. Entities with sequential relationships are grouped closely, following a natural, top-to-bottom and left-to-right reading flow. For modular relationships, elements belonging to the same module are spatially grouped based on the Gestalt principle of proximity, and the grouping is visually reinforced by a bounding box (D2). Due to their similar visual representation, annotative relationships are treated as a special type of modular relationship during layout.

The optimization process begins with the construction of logical constraints. Entities with sequential relationships are positioned in proximity, following a top-to-bottom and left-to-right flow that aligns with natural reading patterns. For modular relationships, elements belonging to the same module are spatially grouped according to the Gestalt principle of proximity, often enclosed by a bounding box to reinforce the grouping structure (D2) visually. Given their analogous visual representation, annotative relationships are treated as a specialized form of modular relationship during layout.

Beyond semantic fidelity, the spatial arrangement must satisfy multiple perceptual criteria. Through a systematic analysis of our collected dataset, we identify key design features, distill empirical knowledge, and abstract these into three optimization objectives: blank space minimization, visual coherence, and bounding-box geometry constraints (D3), which are formally modeled as follows:

Blank Space Minimization aims to maximize space utilization efficiency by encouraging a more compact arrangement of visual elements. A tighter layout reduces wasted space and increases information density, leading to more effective visual communication.

$$Square = 1 - \frac{\sum_{i=1}^{m} S(v_i)}{S\left[Minbox(V)\right]} \tag{1}$$

Formally, this goal is achieved by minimizing the area of the overall layout's bounding box, as defined by the function $Minbox(\cdot)$, where $V = [v_1, v_2, ..., v_m]$ represents the set of visual elements, and $S(\cdot)$ calculates its area.

Visual coherence describes the harmony among visual elements (e.g., icons, typography, color, shape, and layout) within an infographic. This collectively facilitates a seamless and intuitive reading experience by eliminating abrupt transitions and minimizing visual clutter. Visual Information Flow (VIF), following the concept formalized by Lu et al. (Lu et al., 2020), refers to the underlying semantic structure that connects graphical elements to convey information and narrative. It is guided by the strategic arrangement of elements to intuitively direct the viewer along a predetermined visual path, thereby ensuring a logical narrative and an uninterrupted visual progression. To preserve the logical progression of the VIF, we introduce a **Visual Information Flow Disruption Score**, which imposes a penalty when the angle between two consecutive directional elements exceeds 90 degrees (i.e., cosine of the angle < 0), thereby quantifying the disruption to the flow's natural continuity caused by such

sharp turns.

$$VIF = \frac{\sum_{i=1,j=1}^{n} \left| cos(L_i, L_j) \right|}{n}, cos(L_i, L_j) < 0 \quad (2)$$

Here, n represents the number of lines, and $L = \left[L_1, L_2, ..., L_n\right]$ denotes the set of relationship lines.

Bounding-box Geometry Constraint requires that the overall layout conform to a specified outer contour. This ensures the layout maintains shape consistency and adheres to predefined boundaries while satisfying aesthetic and functional requirements.

$$D_{abs} = \left| 1 - \frac{1}{\left| \frac{L[Minbox(V)]}{W[Minbox(V)]} - \frac{L_0}{W_0} \right| + 1} \right| \quad (3)$$

The functions $L(\cdot)$ and $W(\cdot)$ calculate the length and width of the minimum bounding box, respectively, and L_0/W_0 represents the preset optimal aspect ratio.

As outlined in Section 3.2.1, the layout design space is standardized by defining a set of representative basic layouts, with diverse layout variants generated through their recursive combination. Consequently, the information extracted in Section 4.2 is used as input, framing the layout optimization of visual elements as a multi-objective optimization and decision-making problem. The optimization is performed under the aforementioned objective functions, and the final optimized layout solution is returned.

4.3.2. Connection routing optimization

Having established the optimal element arrangement, we proceed to optimize the routing of the connections. A hard constraint in this process is that lines must not pass through the bounding areas of any entity elements. Under this constraint, our strategy aims to minimize both the number of bends in the polylines and the number of connection crossings.

While the connection routing task can be abstracted as a path planning problem, conventional algorithms such as A* (Hart et al., 1968) are unsuitable for our scenario. These algorithms typically operate at the pixel level, leading to prohibitive computational costs given the large number of connections and the requirement for real-time interaction in our system. Moreover, they offer limited control over the precise placement of bends, often resulting in aesthetically suboptimal polylines, and require manual specification of start and end points for each connection. These limitations make existing path planning algorithms inadequate for our application. To address these challenges, we propose a novel algorithm named MultiRect-CenterLine, designed to achieve the following objectives:

- Automated determination of start and end points.
- Elimination of pixel-level searches for significant computational efficiency.
- Shared start/end points for lines on the same rectangle side.
- Minimized the number of polyline bend points.

The MultiRect-CenterLine algorithm operates in several stages. First, it partitions the blank space surrounding the arranged elements into a set of maximum-area rectangles, recording both the blank rectangles adjacent to each element and the adjacency relationships between all such rectangles. The algorithm then traverses this relational graph to identify a connecting path of blank rectangles between any two elements that require a connection, prioritizing the path that traverses the fewest rectangles. Subsequently, the centerline within this identified path of blank rectangles is computed to define the precise coordinate path for the connection. For each potential connection path, an energy value is calculated. This energy value increases when a connection's start and end points coincide and decreases when connections cross. Finally, the algorithm selects the connection configuration with the highest energy value as the optimized routing solution. The detailed procedure is outlined in Algorithm 1.

Algorithm1: Multi-Rect-Centerline

```
/* periodically verify node n's broker list and topic lists */
Define:
    Entities: the size and location of the entity rectangle
    Relationship: the entity relationship that the line needs to express
    find_blank_areas: return of rectangular division of background blank space
    find_common_elements: return of common adjacent blank rectangle
    find_adjacent_blanks: return of adjacent blank area
    find_common_black_edge: return of the right starting point and end point
    find_intersection: return of all the inflection point positions on the line
    get_perpendicular: return of vertical line which passes through the point and the edge
    check_intersection_with_set: return cross-point deduction
Algorithm:
blank_areas ← find_blank_areas(entities)
blank_areas_intersect ← is_intersect(blank_areas)
entity_Intersect ← find_adjacent_blanks(entities,blank_areas_intersect)
For relation in relationship
    blank_area ← find_common_elements(relation,entity_Intersect)
    If exist(start_points,goal_points) and on the lines and find_intersection then
        start,goal ← exist(start_points,goal_points)
        energy ← len(start_points)+len(goal_points)
    Else
        start,goal ← find_common_black_edge(blank_area)
    per_lines ← get_perpendicular(Start,goal,blank_area)
    line_points ← find_intersection(per_lines)
    energy += check_intersection_with_set(line_points ,result)
    If energy > key_energy then
        Result.append(line_points)
        key_energy ← energy
End
```

To systematically evaluate the performance of our proposed MultiRect-CenterLine algorithm, we benchmarked it against two mainstream, efficiency-optimized path planning algorithms: the classic A algorithm (Hart et al., 1968) and its advanced variant, LLM-A* (Meng et al., 2025), which an LLM augments.. The A* algorithm, a widely adopted benchmark, guarantees path optimality by combining heuristic guidance with cost evaluation during graph search. In contrast, LLM-A* represents a state-of-the-art approach that leverages an LLM to enhance heuristic reasoning, enabling

Table 1
Performance Comparison: The Proposed MultiRect-CenterLine Algorithm versus Mainstream Methods

Methodology	Operation	Time (s)	Path-length
A*	164.50	1.02	34.31
LLM-A*	35.33	3.12	34.31
MultiRect-CenterLine	5.33	0.03	40.00

more intelligent search guidance and reduced unnecessary node expansions.

All experiments were conducted on identical hardware using a common set of path planning tasks. Key performance metrics included the total number of operations, computation time (in milliseconds), and final path length.

By employing a spatial partitioning strategy, the MultiRect-CenterLine algorithm avoids pixel-level search, transforming the problem from fine-grained pixel-level scanning to logical reasoning about macro-level rectangular regions and the utilization of their centerlines(Fig. 6). This approach drastically reduced the average number of operations to 5.33 and the average computation time to 0.03 ms, demonstrating a substantial efficiency improvement over mainstream path planning algorithms (Table 1).

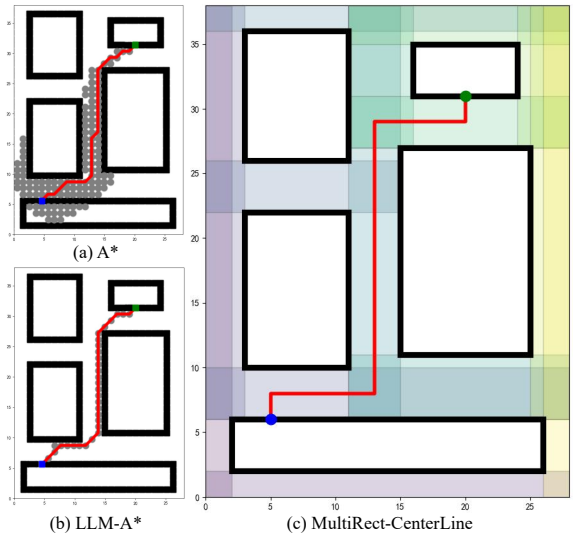


Fig. 6: Visual comparison of path planning processes among A*, LLM-A*, and the proposed MultiRect-CenterLine algorithm.

4.3.3. Iterative interaction

Building on this foundation, we introduce a hybrid optimization strategy that translates user intent into interactive operations, establishing an iterative process centered on three core interactions: Arrangement, Scaling, and Drawing (Fig. 7). These interactions are modeled as constraint formulas and integrated into the optimization algorithm. This integration ensures that manual adjustments remain effective across iterations and maintains a balance between automated optimization and manual control, preventing overrides. By combining automated layout with manual interaction, this hybrid iterative strategy achieves an effective human-AI collaborative optimization process (D4).

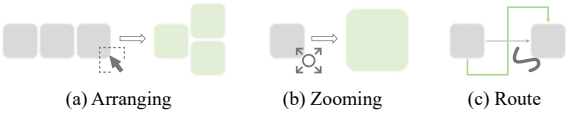


Fig. 7: Interactive operations.

4.4. Visual Elements Optimizer

The visual elements optimizer recommends stylistic schemes based on the optimized layout. Although prior research has focused on generating effective style combinations (Qiu et al., 2022), a key challenge remains in precisely applying these schemes to all elements within a pipeline diagram. The same scheme can yield vastly different visual outcomes when applied to different layouts (Yuan et al., 2022). To overcome this, we construct a style vector library that stores the layout and style attributes of diagram elements in a tree structure. This enables the system to recommend the style scheme most compatible with a given layout through structure-based similarity retrieval.

4.4.1. Constructing the style vector library

The style vector library is constructed by first extracting style features from a collection of pipeline diagrams. These features include element fill color, stroke width, stroke color, text color, and other visual effects (e.g., outer glow, shadow). The features are recorded in a tree structure (Fig. 8), whose hierarchy mirrors that of the pipeline. The root node represents the entire diagram's background, typically set to null. Subsequent nodes are represented in a rectangle + text format: the rectangle records the styling attributes of the entity's visual form, while the text records its textual characteristics; if no text is present, this field is null. Parent-child relationships between nodes express the modular relationships among the entities. Furthermore, if two nodes are identical in content—meaning the current node and all its descendant nodes share the same styling and textual content—they are consolidated into a single node for representation.

Next, inspired by the descriptive vocabulary of NL2Color (Shi et al., 2024), we annotate the extracted styles with textual descriptors (e.g., "soft," "vibrant," "industrial," "professional"). Finally, a word embedding model is employed to convert these textual annotations into vector representations for downstream processing. Separately, the collected icons

Fig. 8: Building the style vector library from pipeline diagrams through a hierarchical tree.

are stored in an icon library, each annotated with a textual label. These labels are converted into vector representations using the same word embedding model mentioned previously. The vectorized style trees and icon vectors together constitute the style vector library, serving as an external knowledge base for the LLM.

4.4.2. Retrieval-based recommendation

First, the hierarchical structure of the target pipeline diagram is represented as a tree. The tree kernel algorithm computes the structural similarity between this tree and those in the style vector library to recommend the most structurally compatible style. An LLM is then employed to refine this initial recommendation. Leveraging the semantic hierarchical relationships within the diagram, the LLM adapts and completes the styles for nodes where the structural match is insufficient or inconsistent (D2, D3). Simultaneously, the system performs shape matching for entities of the same type based on their semantic information. For generic entities, it searches the icon library to recommend icons with semantically consistent features. Users can also upload bitmap images to specify particular entities. These steps collectively yield a visually optimized result.

Building upon this output, users can interactively adjust through textual feedback (e.g., "I want a more professional and rigorous style") (D4). This input is converted into a vector using the same word embedding model. The LLM then queries the style vector library and recommends a new style scheme based on this semantic vector.

5. Evaluation

We evaluated the practicality and effectiveness of PaperDraw through a three-stage process: a case study from the creator's perspective, a user study from the reader's perspective, and a comparative study that benchmarked it against mainstream LLM-based diagram generation methods.

5.1. Case Study

To elucidate the operational mechanism of our method, this section details the process of creating a scientific publications pipeline diagram from a creator's perspective, using a concrete case study. We demonstrate how users can incorporate their creative intent through intuitive, preset interactions, analyzing PaperDraw's performance across three dimensions: practicality, efficiency, and flexibility.

First, the user inputs descriptive text outlining the methodological workflow and clicks "Analyze." The LLM then generates questions, and the user responds by selecting answers (Area ① in Fig. 9), thereby assisting the model in comprehending the text and capturing user intent. Following this, the extracted logical sequences, weight information, and modular structure are visualized on the interface using a predefined code framework (Area ② in Fig. 9), enabling the user to correct inaccuracies through direct interaction.

The system then automatically generates an initial pipeline diagram layout based on the acquired information (Area ③ in Fig. 9). This layout adheres to semantic constraints and satisfies hard constraints, such as preventing lines from crossing entity boundaries. It also seeks a balance among multiple optimization objectives, including spatial efficiency, the aspect ratio of the bounding contour, and visual coherence. Users can interactively refine this layout. For instance, if a user is dissatisfied with the arrangement of entities, they can invoke the "Select" interaction, choose a specific module, and the interface will present alternative layout options recommended by the system. Based on this selection, the system regenerates the layout through a global adjustment. Similarly, to alter the path of a specific connection, a user can employ the "Connection " interaction to modify its start and end points. The system subsequently and automatically recalculates the path according to these new anchor points while respecting all constraints.

Once satisfied with the layout, the user can proceed to style modifications. Initially, the user selects from the style schemes recommended in the right-hand panel (Area ④ in Fig. 9). If the recommended styles are unsatisfactory, the user can input a natural language command (e.g., "Make the style more academic") into the text box within the panel, prompting the LLM to generate new style recommendations based on this instruction.

We recruited six participants to perform the entire workflow. To analyze the efficiency support provided by PaperDraw during the creative process, we recorded the time consumed at each stage (Fig. 10). Following the experiment, a post-experiment evaluation was conducted where participants assessed PaperDraw based on its usability, usefulness, and user satisfaction. They reported that manually creating a pipeline diagram of comparable quality typically required over 30 minutes, whereas using PaperDraw achieved a similar result in approximately three minutes. One user noted, "The interaction methods in PaperDraw are simple and efficient, and it also supports flexible, personalized editing based on the generated result," highlighting the system's pronounced advantage in editability.

5.2. User Study

We compared the pipeline diagrams generated by PaperDraw with baseline diagrams sourced from high-impact journals, including CVPR and TVCG. The comparison was conducted across three dimensions: Information (factual overlap), Semantics (visual coherence and emphasis extraction), and Visual attributes (spatial redundancy and color harmony).

5.2.1. Participants and procedures

We recruited twelve participants (six male, six female) from diverse academic backgrounds, all with extensive experience in writing research papers and creating pipeline diagrams. Each participant completed two rating tasks: (1) describing the core focus of a pipeline in one sentence and then rating its ability to highlight key points based on that description; and (2) rating the color harmony of two sets of pipeline diagrams and completing a 7-point Likert scale questionnaire. To mitigate bias, the diagrams were presented

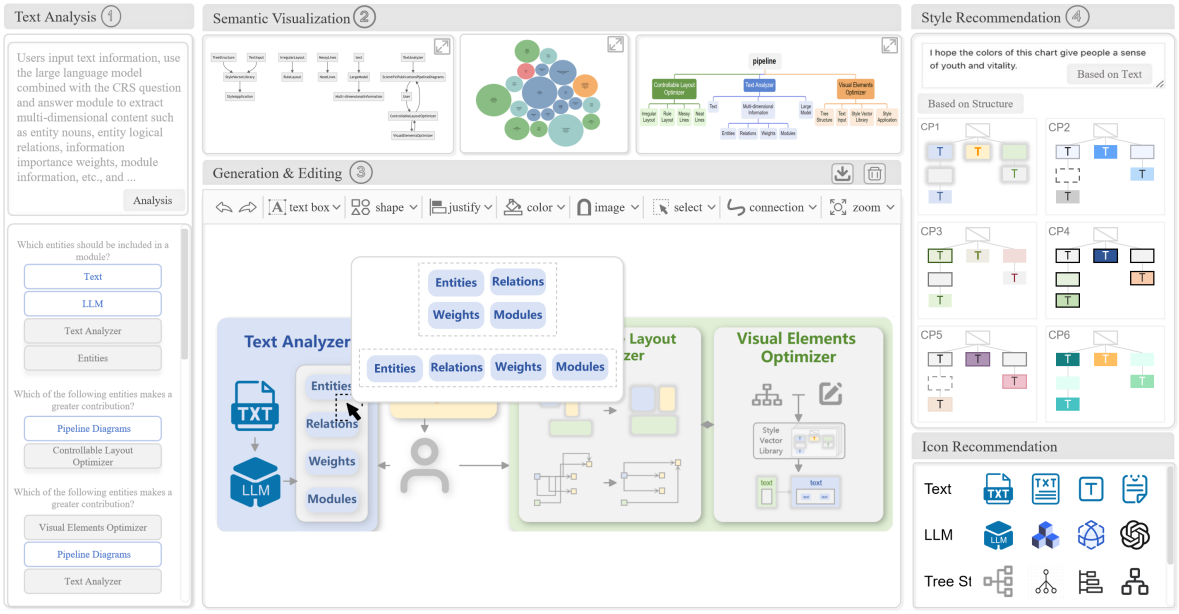


Fig. 9: System User Interface. Text Analysis View(①), dedicated to text input and interactive system Q&A; Semantic Visualization View(②), presenting logical sequences, entity weights, and modular relationships extracted by the LLM; Generation & Editing View(③), for displaying the system-generated diagram and enabling layout adjustments; and Style Recommendation View (④), for applying system-recommended styles or generating new ones via natural language.

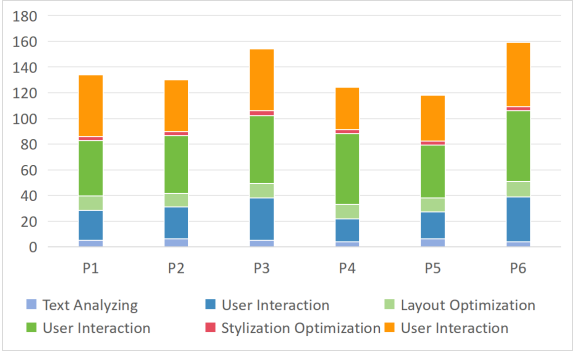


Fig. 10: Efficiency Analysis of the PaperDraw Workflow. The results from six participants (P1-P6) show the duration of both automated system modules and user interactions at each stage.

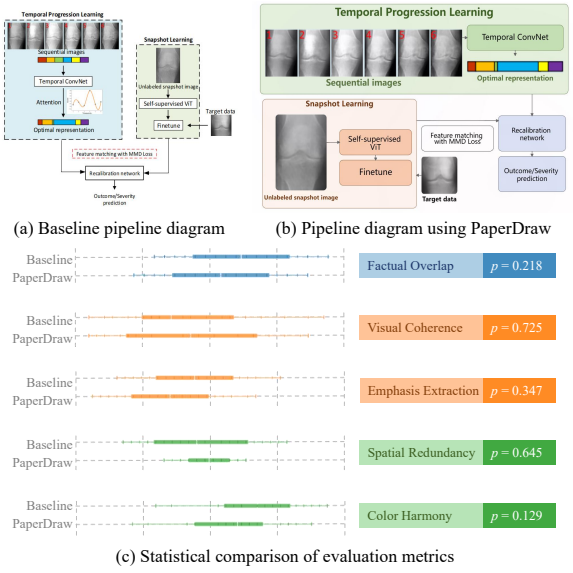


Fig. 11: Exemplar comparison and multi-dimensional evaluation. (a, b) A pair of representative diagrams from the baseline and PaperDraw. (c) Quantitative comparison between the baseline and PaperDraw.

randomly, and participants evaluated them without knowing whether they were generated by PaperDraw or sourced from journals.

In addition to these subjective ratings, we conducted an objective evaluation using three quantitative metrics: factual overlap, visual coherence, and spatial redundancy.

5.2.2. Data analysis

We employed the Mann-Whitney U test for statistical comparison. The results, shown in Fig. 11, indicate that although PaperDraw's scores were marginally lower than the baseline in factual overlap, emphasis extraction, and color harmony, the differences were not statistically significant (all $p > 0.05$). Conversely, PaperDraw demonstrated significantly superior stability in controlling spatial redundancy.

5.3. Comparative Experiment

5.3.1. Baselines for comparison

We systematically compared PaperDraw against three categories of baseline methods: (i) XML Code Generation, which prompts an LLM to generate an XML file that defines the diagram's structure and hierarchy, subsequently parsed and rendered into a visual graphic by a dedicated tool; (ii)

Table 2
Performance Comparison: PaperDraw vs. Existing LLM-Based Pipeline Diagrams Generation Methods

Model	Information Score			Visual Score				Flexibility	Efficiency & Cost	
	Content	Clarity	Overall	Color	Element	Layout	Overall	Editable	Token	Time (s)
GPT-5 xml	3.26	3.23	3.25	3.81	3.78	4.01	3.87	4.00	5016.00	31.00
DS-R1 xml	3.73	3.30	3.52	4.14	4.20	3.80	4.05	4.00	6738.00	166.00
GPT-5 images	2.18	3.02	2.60	**4.50**	4.05	4.11	4.22	1.00	1712.00	107.00
DS-R1 images	2.57	2.34	2.46	3.01	3.54	3.87	3.47	1.00	2210.00	87.00
GPT-5+Draw.io	3.25	3.58	3.42	4.11	4.05	3.58	3.91	4.00	**1337.00**	**9.70**
DS-R1+Draw.io	4.23	3.47	3.85	4.02	3.87	3.98	3.96	4.00	5382.00	203.00
GPT-5+paperdraw	4.28	**4.22**	4.25	**4.50**	4.21	4.23	4.31	**4.25**	5378.00	56.00
DS-R1+paperdraw	**4.29**	4.14	**4.22**	4.28	**4.31**	**4.43**	**4.34**	**4.25**	6723.00	171.00

Direct Image Generation, which refers to LLM-based, end-to-end image synthesis; and (iii) Draw.io Integrated Generation, where an LLM produces a structured description compliant with the Draw.io specification, which is then imported into the Draw.io platform to create the diagram.

The experiment utilized two widely recognized, high-performance LLMs, DeepSeek-R1 and GPT-5, as the foundational generators. All methods were evaluated under the same framework to ensure a fair comparison.

5.3.2. Quantitative comparative analysis

As shown in Table 2, a comprehensive multi-metric comparison with existing LLM-based image generation methods shows that PaperDraw holds significant advantages in information delivery, visual quality, and editability.

Information Delivery Capability: Methods leveraging XML generation or Draw.io integration demonstrated strong performance in content completeness and clarity. PaperDraw, however, excelled in overall information delivery. This is attributed to its interactive intent capture and structured text comprehension. Notably, end-to-end image generation methods (e.g., GPT-5 images and DS-R1 images) underperformed on information-centric metrics, highlighting an inherent limitation in preserving semantic accuracy through purely visual generation.

Visual Expression Quality: End-to-end image generation methods (e.g., GPT-5 images and DS-R1 images) achieved high visual scores, owing to their training on extensive image datasets. Structure-based methods (i.e., XML and Draw.io) demonstrated balanced color, layout, and element design performance. Supported by its specialized connection routing algorithm and style vector library, PaperDraw consistently achieved high performance across all visual dimensions.

Editability: XML-based and Draw.io-integrated methods received high editability ratings, benefiting from their explicit structural representations and modular designs. PaperDraw further enhanced usability and customizability via its layout optimization algorithms and intelligent style recommendation mechanism. In contrast, end-to-end image generation methods scored significantly lower because their outputs lack an editable structure, which severely limits their applicability in scenarios requiring iterative revisions.

Efficiency and Cost: The combination of GPT-5 and Draw.io proved to be the lowest-cost solution, underscoring the efficiency of tool-assisted methods. PaperDraw maintained high-quality output with moderate computational and time costs, striking a favorable balance between practical expense and result quality.

5.3.3. Qualitative comparative analysis

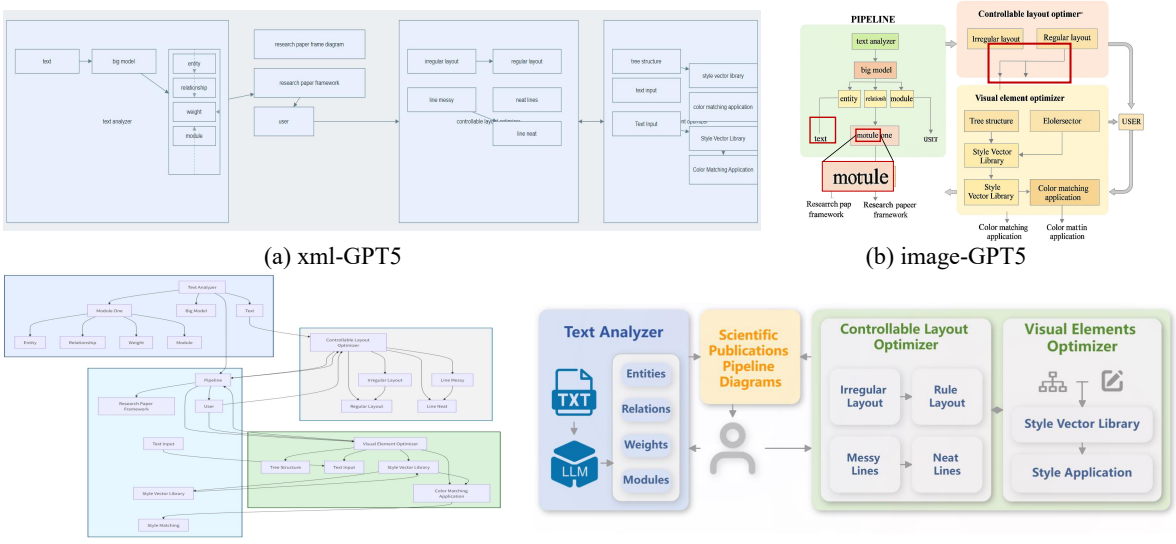
Fig. 12 compares the scientific publication pipeline diagram generated from the same text input using GPT-5 across different baseline methods and the proposed PaperDraw approach. As shown in Fig. 12(a) and Fig. 12(c), methods utilizing XML and Draw.io leverage their structured language characteristics to produce diagrams with rigorous logic and clear hierarchy, demonstrating strong standardization and semantic consistency. However, due to the lack of explicit visual design guidance, these methods exhibit significant deficiencies in overall layout aesthetics and visual style.

In contrast, end-to-end image generation methods are advantageous in visual presentation, often producing more aesthetically appealing results, as seen in Fig. 12(b). Nevertheless, their limitations are pronounced. The generated graphics are typically not editable, restricting their practical flexibility and scalability. Moreover, the outputs frequently contain errors, such as textual inaccuracies, local blurring, missing connections, or misaligned elements, resulting in poor robustness and accuracy.

The proposed PaperDraw effectively integrates the strengths of the approaches above. It decouples the generation process into two distinct stages: structured semantic construction and visual optimization. This strategy allows the LLM to focus on its semantic reasoning and logical organization strengths while delegating style control and spatial layout to deterministic algorithms. This collaborative mechanism not only enhances the reliability and visual consistency of the outputs but also retains considerable flexibility and customizability, thereby achieving a superior balance between generation quality and editability.

6. Conclusion

We present PaperDraw, an interactive system that automatically generates publication-quality pipeline diagrams from natural language descriptions. By integrating a large



(a) xml-GPT5 (b) image-GPT5

(c) Draw.io-GPT5 (d) PaperDraw-GPT5

Fig. 12: Qualitative comparison of pipeline diagrams generated by different GPTS-based methods.

language model with a controllable layout algorithm and a style vector library, PaperDraw produces academic-style layouts while offering diverse visual recommendations. The system supports flexible customization and iterative refinement, significantly reducing manual effort. Through case studies, user studies, and comparative experiments, we demonstrate that PaperDraw enhances efficiency over manual design and outperforms existing intelligent generation methods in both logical accuracy and visual quality. Our work contributes to the growing field of AI for Research(Chen et al., 2025), particularly AI for Academic Writing.

CRediT authorship contribution statement

Xin Li: Writing – review and editing, Funding acquisition, Project administration, Conceptualization, Supervision. **Jixiu Liao:** Writing - original draft, Writing – review and editing, Investigation, Methodology, Validation, Data curation. **Chenglin Xu:** Software, Visualization. **Xianzheng Pang:** Investigation, Validation.

Ethical Approval

This study does not contain any studies with human or animal subjects performed by any of the authors.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was supported by the Shandong Provincial Natural Science Foundation, China (Grant No. ZR2024MF048).

References

Alrwele, D.N.S., 2017. Effects of Infographics on Student Achievement and Students' Perceptions of the Impacts of Infographics. Journal of Education and Human Development (JEHD) 6, 104–117. doi:10.15640/jehd.v6n3a12.

Chen, Q., Yang, M., Qin, L., Liu, J., Yan, Z., Guan, J., Peng, D., Ji, Y., Li, H., Hu, M., Zhang, Y., Liang, Y., Zhou, Y., Wang, J., Chen, Z., Che, W., 2025. AI4Research: A Survey of Artificial Intelligence for Scientific Research. arXiv:2507.01903. arXiv preprint.

Cheng, Z., Lin, Z., Yang, Y., Wei, Z., Chen, S., 2025. Interactive simulation and visual analysis of social media event dynamics with LLM-based multi-agent modeling. Visual Informatics 9, 100260. doi:10.1016/j.visinf.2025.100260.

Clark, J.M., Paivio, A., 1991. Dual coding theory and education. Educ Psychol Rev 3, 149–210. doi:10.1007/BF01320076.

Cui, W., Zhang, X., Wang, Y., Huang, H., Chen, B., Fang, L., Zhang, H., Lou, J.G., Zhang, D., 2020. Text-to-Viz: Automatic Generation of Infographics from Proportion-Related Natural Language Statements. IEEE Trans. Visual. Comput. Graphics 26, 906–916. doi:10.1109/TVCG.2019.2934785.

Desolneux, A., Moisan, L., Morel, J.M., 2004. Gestalt Theory and Computer Vision, in: Carsetti, A. (Ed.), Seeing, Thinking and Knowing. Kluwer Academic Publishers, Dordrecht. volume 38, pp. 71–101. doi:10.1007/1-4020-2081-3_4.

Dibia, V., 2023. Lida: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models. arXiv:2303.02927. arXiv preprint.

Dibia, V., Demiralp, c., 2019. Data2vis: Automatic generation of data visualizations using sequence-to-sequence recurrent neural networks. IEEE Comput. Graph. Appl. 39, 33–46. doi:10.1109/MCG.2019.2924636.

Ehsani, K., Tulsiani, S., Gupta, S., Farhadi, A., Gupta, A., 2020. Use the Force, Luke! Learning to Predict Physical Forces by Simulating Effects. arXiv:2003.12045. arXiv preprint.

Gastel, B., Day, R.A., 2022. How to write and publish a scientific paper. Ninth edition ed., Cambridge University Press.

Gowaikar, S., Iyengar, S., Segal, S., Kalyanaraman, S., 2024. An Agentic Approach to Automatic Creation of P&ID Diagrams from Natural Language Descriptions. arXiv:2412.12898. arXiv preprint.

Harper, J., Agrawala, M., 2014. Deconstructing and Restyling D3 Visualizations, in: Proceedings of the 27th annual ACM symposium on User interface software and technology, ACM, Honolulu Hawaii USA. pp. 253–262. doi:10.1145/2642918.2647411.

Hart, P., Nilsson, N., Raphael, B., 1968. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Trans. Syst. Sci. Cyber. 4, 100–107. doi:10.1109/TSSC.1968.300136.

He, Y., Xu, K., Cao, S., Shi, Y., Chen, Q., Cao, N., 2025. Leveraging foundation models for crafting narrative visualization: A survey. IEEE Transactions on Visualization and Computer Graphics 31, 9303–9323. doi:10.1109/TVCG.2025.3542504.

Head, A., Lo, K., Kang, D., Fok, R., Skjonsberg, S., Weld, D.S., Hearst, M.A., 2021. Augmenting Scientific Papers with Just-in-Time, Position-Sensitive Definitions of Terms and Symbols, in: Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems, ACM, Yokohama Japan. pp. 1–18. doi:10.1145/3411764.3445648.

Horita, D., Inoue, N., Kikuchi, K., Yamaguchi, K., Aizawa, K., 2024. Retrieval-augmented layout transformer for content-aware layout generation. arXiv:2311.13602. arXiv preprint.

Inoue, N., Kikuchi, K., Simo-Serra, E., Otani, M., Yamaguchi, K., 2023a. Layoutdm: Discrete diffusion model for controllable layout generation. doi:10.1109/TVCG.2025.3542504, arXiv:2303.08137. arXiv preprint.

Inoue, N., Kikuchi, K., Simo-Serra, E., Otani, M., Yamaguchi, K., 2023b. Layoutdm: Discrete diffusion model for controllable layout generation. arXiv:2303.08137. arXiv preprint.

Kong, X., Jiang, L., Chang, H., Zhang, H., Hao, Y., Gong, H., Essa, I., 2022. BLT: Bidirectional Layout Transformer for Controllable Layout Generation, in: Avidan, S., Brostow, G., Cissé, M., Farinella, G.M., Hassner, T. (Eds.), Computer Vision – ECCV 2022. Springer Nature Switzerland, Cham. volume 13677, pp. 474–490. doi:10.1007/978-3-031-19790-1_29.

Kulkarni, A., Shah, H., D'Mello, L., Shah, K., 2023. Flowchart Generation and Mind Map Creation using Extracted Summarized Text, in: 2023 International Conference on Recent Advances in Science and Engineering Technology (ICRASET), IEEE, B G NAGARA, India. doi:10.1109/ICRASET59632.2023.10420315.

Lee, B., Riche, N.H., Isenberg, P., Carpendale, S., 2015. More Than Telling a Story: Transforming Data into Visually Shared Stories. IEEE Comput. Grap. Appl. 35, 84–90. doi:10.1109/MCG.2015.99.

Li, J., Yang, J., Hertzmann, A., Zhang, J., Xu, T., 2021. LayoutGAN: Synthesizing Graphic Layouts With Vector-Wireframe Adversarial Networks. IEEE Trans. Pattern Anal. Mach. Intell. 43, 2388–2399. doi:10.1109/TPAMI.2019.2963663.

Li, Z., Zhang, W., Tao, J., 2025. FlowLLM: Large language model driven flow visualization. Visual Informatics 9, 100241. doi:10.1016/j.visinf.2025.100241.

Lu, M., Wang, C., Lanir, J., Zhao, N., Pfister, H., Cohen-Or, D., Huang, H., 2020. Exploring Visual Information Flows in Infographics, in: Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems, ACM, Honolulu HI USA. pp. 1–12. doi:10.1145/3313831.3376263.

Meng, S., Wang, Y., Yang, C.F., Peng, N., Chang, K.W., 2025. LLM-A*: Large Language Model Enhanced Incremental Heuristic Search on Path Planning. arXiv:2407.02511. arXiv preprint.

Merkelbach, S., Heuwinkel, T., Dumitrescu, R., 2024. PID-Gen: Towards an Algorithm for the Generation of Random P&IDs, in: 2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA), IEEE, Padova, Italy. pp. 1–4. doi:10.1109/ETFA61755.2024.10710643.

Moritz, D., Wang, C., Nelson, G.L., Lin, H., Smith, A.M., Howe, B., Heer, J., 2019. Formalizing Visualization Design Knowledge as Constraints: Actionable and Extensible Models in Draco. IEEE Trans. Visual. Comput. Graphics 25, 438–448. doi:10.1109/TVCG.2018.2865240.

Qiu, Q., Otani, M., Iwazaki, Y., 2022. An Intelligent Color Recommendation Tool for Landing Page Design, in: 27th International Conference on Intelligent User Interfaces, ACM, Helsinki Finland. pp. 26–29. doi:10.1145/3490100.3516450.

Setlur, V., Battersby, S.E., Tory, M., Gossweiler, R., Chang, A.X., 2016. Eviza: A Natural Language Interface for Visual Analysis, in: Proceedings of the 29th Annual Symposium on User Interface Software and Technology, ACM, Tokyo Japan. pp. 365–377. doi:10.1145/2984511.2984588.

Shi, C., Cui, W., Liu, C., Zheng, C., Zhang, H., Luo, Q., Ma, X., 2024. Nl2color: Refining color palettes for charts with natural language. IEEE Transactions on Visualization and Computer Graphics 30, 814–824. doi:10.1109/TVCG.2023.3326522.

Shi, D., Sun, F., Xu, X., Lan, X., Gotz, D., Cao, N., 2021a. AutoClips: An Automatic Approach to Video Generation from Data Facts. Computer Graphics Forum 40, 495–505. doi:10.1111/cgf.14324.

Shi, D., Xu, X., Sun, F., Shi, Y., Cao, N., 2021b. Calliope: Automatic Visual Data Story Generation from a Spreadsheet. IEEE Trans. Visual. Comput. Graphics 27, 453–463. doi:10.1109/TVCG.2020.3030403.

Shixia Liu, Yingcai Wu, Enxun Wei, Mengchen Liu, Yang Liu, 2013. StoryFlow: Tracking the Evolution of Stories. IEEE Trans. Visual. Comput. Graphics 19, 2436–2445. doi:10.1109/TVCG.2013.196.

Tang, T., Li, R., Wu, X., Liu, S., Knittel, J., Koch, S., Ertl, T., Yu, L., Ren, P., Wu, Y., 2021. PlotThread: Creating Expressive Storyline Visualizations using Reinforcement Learning. IEEE Trans. Visual. Comput. Graphics 27, 294–303. doi:10.1109/TVCG.2020.3030467.

Tenopir, C., King, D.W., Edwards, S., Wu, L., 2009. Electronic journals and changes in scholarly article seeking and reading patterns. Aslib Proceedings 61, 5–32. doi:10.1108/00012530910932267.

Wang, Y., Hou, Z., Shen, L., Wu, T., Wang, J., Huang, H., Zhang, H., Zhang, D., 2023. Towards natural language-based visualization authoring. IEEE Transactions on Visualization and Computer Graphics 29, 1222–1232. doi:10.1109/TVCG.2022.3209357.

Wu, G., Guo, S., Hoffswell, J., Chan, G.Y.Y., Rossi, R.A., Koh, E., 2024. Socrates: Data story generation via adaptive machine-guided elicitation of user feedback. IEEE Transactions on Visualization and Computer Graphics 30, 131–141. doi:10.1109/TVCG.2023.3327363.

Yuan, L.P., Zhou, Z., Zhao, J., Guo, Y., Du, F., Qu, H., 2022. Infocolorizer: Interactive recommendation of color palettes for infographics. IEEE Transactions on Visualization and Computer Graphics 28, 4252–4266. doi:10.1109/TVCG.2021.3085327.

Zhou, K., Cai, R., Ma, Y., Tan, Q., Wang, X., Li, J., Shum, H.P.H., Li, F.W.B., Jin, S., Liang, X., 2023. A Video-Based Augmented Reality System for Human-in-the-Loop Muscle Strength Assessment of Juvenile Dermatomyositis. IEEE Trans. Visual. Comput. Graphics 29, 2456–2466. doi:10.1109/TVCG.2023.3247092.

Zhu, S., Sun, G., Jiang, Q., Zha, M., Liang, R., 2020. A survey on automatic infographics and visualization recommendations. Visual Informatics 4, 24–40. doi:10.1016/j.visinf.2020.07.002.